

Corporación Universitaria Iberoamericana



Actividad 4: Herramientas y tecnologías de calidad de software

03 octubre de 2026

Herramientas de calidad de software aplicadas al caso Therac-25

Introducción

Según Aizprua et al. (2019), el concepto de calidad del software debe reinterpretarse desde la perspectiva de una actividad continua en relación con todos los procesos de desarrollo, en lugar de como una tarea específica que solo requiere la realización cuando el proyecto se lleva a completar. Además, el concepto coincide con la definición de calidad de los sistemas de información dada por Piattini-Velthuis y García-Rubio (2015). Del mismo modo, Pressman y Maxim (2021) argumentan que el aseguramiento de la calidad cubre un conjunto de actividades diseñadas para proporcionar pruebas de conformidad de un producto con requisitos establecidos. Las actividades del aseguramiento de la calidad implican el uso de herramientas específicas desarrolladas para automatizar el proceso de pruebas, analizar código y realizar un seguimiento de cualquier posible error.

Por lo tanto, para este informe, se repasa el caso descrito en el Trabajo Práctico N.º 2 el accidentador médico lineal de Therac-25. El software utilizado en 25 Therac sufrió diversas fallas entre 1985 y 1987, lo que causó a los pacientes una dosis excesiva de irradiación; como resultado, hubo muertes (Leveson y Turner, 1993). A raíz del problema, se analizarán herramientas de calidad actuales, comparando y contrastando sus características clave y evaluando cuál puede ayudar a mejorar el sistema, basándose también en la metodología de las instrucciones del curso. Luego, se ofrecerá una descripción de esta herramienta y de su aplicación a un escenario de casos similar al de Therac-25 para detectar cualquier irregularidad y prevenir nuevos errores o desviaciones del sistema.

El informe seguirá la secuencia descrita en las instrucciones de la actividad: comenzará por revisar herramientas de calidad y describir sus aplicaciones y, luego, tras comparar e identificar la herramienta potencial y sus fortalezas, se describirá y aplicará a continuación junto con sus beneficios y recomendaciones.

Objetivos

Objetivo general. Seleccionar y aplicar de forma descriptiva una herramienta de calidad de software sobre el caso Therac-25, evaluando su impacto en la prevención de las fallas identificadas.

Objetivos específicos.

- Investigar y comparar herramientas de pruebas automatizadas, análisis estático y gestión de pruebas.
- Justificar la selección de una herramienta a partir de sus características, funcionalidades y ventajas.
- Describir su aplicación sobre un módulo simulado equivalente al del caso y proponer medidas preventivas.

1. Caso de estudio: el Therac-25

El Therac-25 fue un acelerador lineal de uso médico fabricado por Atomic Energy of Canada Limited (AECL) que ofrecía dos modos de tratamiento: haz de electrones de baja energía y rayos X de alta energía mediante un blanco de tungsteno. Entre junio de 1985 y enero de 1987 se documentaron seis accidentes en los que pacientes recibieron dosis muy superiores a las prescritas, con consecuencias graves y, en varios casos, fatales (Leveson & Turner, 1993).

Según la investigación de Leveson y Turner (1993), los accidentes no tuvieron una causa única, sino que resultaron de la suma de errores técnicos y fallas de proceso. Leveson (1995) añade que muchos accidentes asociados al software provienen de requisitos incorrectos o incompletos y no de simples errores de programación, de manera que confiabilidad y seguridad no son sinónimos. La Tabla 1 resume las fallas principales y las asocia con factores de calidad del modelo de McCall et al. (1977), que agrupa los atributos del producto según su operación, su revisión y su transición, y que Pressman y Maxim (2021) retoman. Vale notar que la versión 2023 de la norma ISO/IEC 25010 incorporó la seguridad operacional (*safety*) como una característica de calidad del producto (International Organization for Standardization [ISO] & International Electrotechnical Commission [IEC], 2023), lo que confirma la relevancia de este atributo en casos como el estudiado.

Tabla 1

Fallas principales del Therac-25 y atributos de calidad afectados

Falla	Descripción	Atributo afectado
F1. Condición de carrera	Si el operador editaba rápidamente los parámetros de tratamiento, la tarea de configuración podía usar datos desactualizados y dejar el equipo en un estado inconsistente.	Fiabilidad, corrección
F2. Desbordamiento de contador	Una variable de un solo byte se incrementaba en cada ciclo; al desbordarse y valer cero, se omitía una verificación de posición del colimador.	Corrección, fiabilidad
F3. Eliminación de interbloqueos por hardware	Se retiraron los enclavamientos físicos del Therac-20 y se confió la seguridad al software.	Integridad, seguridad
F4. Reutilización sin revisión	Se reutilizó software de modelos anteriores cuyos errores estaban enmascarados por los interbloqueos de hardware.	Reusabilidad, mantenibilidad
F5. Mensajes de error crípticos	El equipo mostraba códigos como “MALFUNCTION 54” sin explicación, y los operadores aprendieron a ignorarlos.	Usabilidad

F6. Pruebas y revisión insuficientes	El software no tuvo revisión independiente ni pruebas integrales del sistema completo; la respuesta a los incidentes reportados fue deficiente.	Facilidad de prueba, verificabilidad
--------------------------------------	---	--------------------------------------

Nota. Elaborado a partir de Leveson y Turner (1993), McCall et al. (1977) y Pressman y Maxim (2021).

2. Selección y análisis de herramientas

Se investigaron cinco herramientas representativas de tres categorías de la actividad: análisis estático, pruebas automatizadas y gestión de pruebas. La selección busca cubrir distintos momentos del proceso de calidad y no solo comparar productos similares. Esta mirada amplia es coherente con Myers et al. (2011), quienes entienden la prueba como la ejecución de un programa con la intención de encontrar errores y recomiendan combinar varias técnicas, dado que no es posible probar un programa de forma exhaustiva.

2.1 Herramientas investigadas

Tabla 2

Características, funcionalidades y problemas que aborda cada herramienta

Herramienta	Categoría	Funcionalidades principales	Problemas que puede abordar
SonarQube (SonarSource)	Análisis estático	Analiza el código sin ejecutarlo y reporta bugs, vulnerabilidades, puntos críticos de seguridad y code smells; mide duplicación, complejidad y deuda técnica; aplica <i>quality gates</i> ; se integra con sistemas de CI.	Errores de lógica, uso inseguro de datos, código duplicado o confuso, ausencia de cobertura en código nuevo.
JUnit	Pruebas unitarias (Java)	Marco de pruebas automatizadas con aserciones, ciclo de vida de pruebas y ejecución desde Maven, Gradle o el IDE.	Regresiones y errores en la lógica de unidades pequeñas de código.
Selenium	Pruebas funcionales de interfaz web	Automatiza navegadores para simular la interacción del usuario con aplicaciones web.	Fallas de flujo en interfaces web; no analiza código ni sistemas embebidos.

TestRail	Gestión de pruebas	Organiza casos, planes y ejecuciones de prueba; genera informes de trazabilidad y avance.	Falta de trazabilidad entre requisitos, pruebas y defectos.
SoapUI (SmartBear)	Pruebas de servicios web/API	Crea y ejecuta pruebas funcionales, de carga y de seguridad sobre servicios SOAP y REST.	Defectos en la comunicación entre servicios y contratos de API.

Nota. Elaborado a partir de la documentación oficial de cada herramienta (Campbell & Papapetrou, 2013; JUnit, s. f.; Selenium, s. f.; SmartBear Software, 2024; SonarSource, s. f.).

2.2 Comparación

Para comparar se emplearon los criterios de la actividad (facilidad de uso, costo, soporte de plataformas, integración y comunidad) y se añadió un sexto: la pertinencia respecto al caso Therac-25. Cada criterio se valoró de 1 (bajo) a 5 (alto). La valoración es un juicio razonado del autor a partir de la documentación consultada, no una medición objetiva, y su finalidad es apoyar la decisión de forma transparente.

Tabla 3

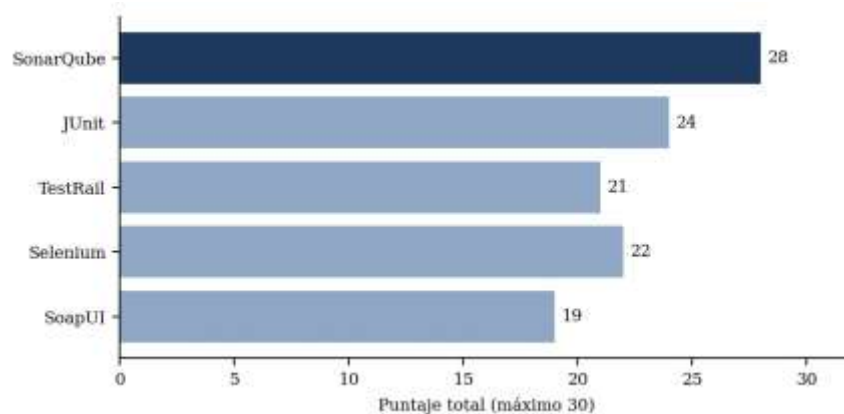
Matriz de comparación (1 = bajo, 5 = alto)

Criterio	SonarQube	JUnit	TestRail	Selenium	SoapUI
Facilidad de uso	4	4	4	3	4
Costo (5 = más accesible)	4	5	2	5	4
Soporte de plataformas y lenguajes	5	3	5	4	4
Integración con el proceso de desarrollo	5	4	4	4	3
Comunidad y soporte	5	5	3	5	3
Pertinencia para el caso Therac-25	5	3	3	1	1
Total (máx. 30)	28	24	21	22	19

Nota. Valoración propia. En costo, SonarQube cuenta con una edición comunitaria gratuita y ediciones comerciales; TestRail es un producto de pago por suscripción; JUnit y Selenium son de código abierto.

Figura 1

Puntaje total por herramienta según la matriz de comparación



Nota. Elaboración propia.

2.3 Ventajas y desventajas según el contexto

- **SonarQube.** Ventaja: detecta defectos tempranamente, de forma automática y repetible, y produce métricas objetivas. Desventaja: genera falsos positivos, requiere configurar reglas y no sustituye las pruebas dinámicas.
- **JUnit.** Ventaja: sencillo y muy extendido para verificar la lógica. Desventaja: solo es tan bueno como los casos escritos y no revela por sí solo problemas de concurrencia.
- **TestRail.** Ventaja: da trazabilidad y control del proceso de pruebas. Desventaja: costo y esfuerzo de mantenimiento, y no analiza el código.
- **Selenium y SoapUI.** Ventaja: excelentes en su nicho (interfaz web y APIs). Desventaja: el caso Therac-25 es un sistema embebido de control, por lo que su aporte directo es limitado.

3. Herramienta seleccionada: SonarQube

La herramienta seleccionada, tras analizar las filas de la matriz de evaluación, es SonarQube, ya que obtuvo la puntuación más alta y ofrece características de análisis estático relevantes para el caso del Therac-25; en particular, cabe destacar que varias de sus «alarmas» están relacionadas con los problemas mencionados en el caso. Para simplificar, el razonamiento que condujo a la selección de SonarQube como herramienta pertinente se basa en mapear los problemas del Therac-25 con las características publicadas de la herramienta. Así, los problemas concretos mencionados en el caso dan lugar a lo siguiente:

Acceso/actualización concurrentes a datos compartidos. En el contexto del Therac-25, uno de los problemas clave detectados —y la razón principal del incendio— se debió a la falta de bloqueo de memoria en el código, que dio lugar a actualizaciones concurrentes de datos compartidos y, por tanto, a condiciones de carrera. Las características de SonarQube permiten detectar ciertos patrones de código peligrosos en nivel de código fuente; por

ejemplo, el uso de variables compartidas en entornos multihilo (dependiendo del lenguaje, podría detectarse). Cabe destacar que esta característica no permite detectar todas las condiciones de carrera (ni siquiera se puede estar seguro de que se hayan detectado algunas), pero es relevante, ya que se trata de una de las causas fundamentales de errores en el Therac-25. Por lo tanto, un análisis estático que detecte ciertas condiciones de carrera podría ser una herramienta pertinente para desarrolladores de software de tiempo real, que deben garantizar la seguridad y la fiabilidad del sistema.

Desbordamiento de contador. El código del Therac-25 también presentaba un problema relacionado con el riesgo de desbordamiento de un contador. En este caso, el uso de la herramienta SonarQube puede ser pertinente, ya que el análisis estático puede detectar operaciones con desbordamiento en el código fuente (con el nivel de detalle adecuado, en función del lenguaje). Por lo tanto, al aplicar dicha herramienta a un módulo del sistema, un análisis estático podría detectar y alertar sobre el riesgo de que se produjera un desbordamiento, permitiendo corregirlo (por ejemplo, modificando el código para comprobar el valor máximo permitido para el contador).

Reutilización de código sin adaptaciones pertinentes. Aquí, el problema que se detectó en el Therac-25 y que también requiere un análisis estático es el uso de código fuente repetido sin «adaptaciones específicas». En el caso del Therac-25, la lógica de software se reutilizó en varios componentes y, al no tener en cuenta la «adhesión» del código a sus requisitos, se produjo un fallo. Este problema podría resolverse utilizando un análisis estático de código para detectar la presencia de código repetido. A continuación, el desarrollador (o la herramienta) podría decidir si modificar el código, teniendo en cuenta las restricciones adicionales del nuevo módulo, o evitar el uso de dicho módulo. Manejo deficiente de errores. Otra causa de fallos en el Therac-25 fue el manejo deficiente de errores en el sistema de software. Algunos de ellos podrían detectarse mediante un análisis estático del código, por ejemplo, cuando una excepción se ignora o no se procesa en su totalidad. En este caso, el sistema podría encontrarse en un estado incorrecto, y la herramienta puede alertar (sobre la base del código) de que existe la posibilidad de que el sistema no se comporte de la manera esperada. Falta de cobertura de las pruebas unitarias. Una de las últimas causas mencionadas en el caso de Therac-25 es la ausencia de cobertura de las pruebas unitarias para el sistema. Aquí, un análisis estático del sistema puede ayudar a identificar la cobertura general de las pruebas, mostrando qué porcentaje (si es posible) del código fuente se ejerce mediante pruebas unitarias. Cabe destacar que el análisis estático aquí solo complementa las pruebas, ya que no puede reemplazar ni proporcionar pruebas de correctitud. Conclusión. Como puede observarse a partir del análisis de las características de la herramienta y de las fuentes del caso, SonarCube es una herramienta relevante para el módulo simulado del Therac-25. Como puede verse, varios de los problemas que se detectaron en el sistema pueden detectarse mediante el análisis estático de su código fuente. Un análisis estático permite detectar a priori patrones peligrosos en el código e incluso aplicar algunos controles automáticos para aumentar la fiabilidad del sistema. Dicho esto, es de conocimiento común que los sistemas críticamente seguros, en particular los sistemas de software, no pueden depender únicamente del análisis estático, los patrones, la automatización y las pruebas unitarias; los aspectos dinámicos, por el contrario, requieren la intervención y la comprobación manual.

3.1 Características y funcionalidades relevantes

Campbell y Papapetrou (2013) explican que SonarQube evalúa el código desde siete ejes de calidad: arquitectura y diseño, comentarios, reglas de codificación, posibles errores, complejidad, pruebas unitarias y duplicaciones. A partir de ese enfoque, sus funcionalidades más pertinentes para el caso son las siguientes.

- **Reglas de calidad y seguridad:** clasifica los hallazgos en bugs, vulnerabilidades, puntos críticos de seguridad y code smells, con indicación de severidad y de cómo corregirlos.
- **Métricas de mantenibilidad:** complejidad, duplicación de código y estimación de deuda técnica, útiles para evaluar el riesgo de reutilizar código heredado. El concepto de deuda técnica fue propuesto por Cunningham (1992) como metáfora de los costos futuros que genera apresurar soluciones de código.
- **Quality gate:** conjunto de condiciones que el código nuevo debe cumplir para considerarse aceptable; si no las cumple, el cambio no avanza.
- **Integración continua:** se ejecuta automáticamente en cada commit o pull request desde sistemas de CI, y muestra el resultado a todo el equipo.
- **Cobertura de pruebas:** importa los reportes de herramientas como JUnit y evidencia qué código no está siendo probado.

3.2 Limitaciones que deben reconocerse

Si bien SonarQube ofrece numerosas ventajas para determinar la calidad del software, también presenta algunas desventajas inherentes que deben tenerse en cuenta a la hora de aplicarlo. Uno de los problemas más evidentes radica en la comparabilidad con el caso real del Therac-25. El software que controlaba la máquina era mucho más antiguo, ya que giraba en torno al lenguaje ensamblador y al procesador PDP-11 (Leveson y Turner, 1993). La aplicación de SonarQube a este caso, por lo tanto, no puede ser completa, ya que el sistema no está diseñado para trabajar con software de ese tipo. Por este motivo, para analizar la calidad del software, se ha creado un módulo simplificado en un lenguaje de programación moderno que reproduce algunas de las características problemáticas del Therac-25. Un problema importante de la técnica de análisis estático consiste en que resulta difícil determinar si se han tenido en cuenta todas las condiciones de carrera aplicables, por lo que es posible que existan algunas que todavía no se hayan tenido en cuenta. Los errores de concurrencia podrían pasar desapercibidos en el análisis estático, aunque probablemente se detectarían mediante pruebas dinámicas. La herramienta seleccionada no puede ser la única solución para garantizar la seguridad del sistema, y los resultados que ofrece deben complementarse con el trabajo de otros inspectores. Es posible que algunos de los resultados detectados por la herramienta seleccionada se deban a falsos positivos o que requieran un análisis adicional en profundidad. Por último, dado que la herramienta solo puede ofrecer información sobre el código fuente, su análisis es insuficiente para comprender cómo funciona el sistema en su conjunto o si se han tenido en cuenta todas las consideraciones de

diseño, como la seguridad o los requisitos. En general, SonarQube solo puede ofrecer información sobre la calidad del software del Therac-25; sin embargo, incluso sus resultados positivos no pueden considerarse una garantía de seguridad. La herramienta podría integrarse en el flujo de trabajo de desarrollo junto con las pruebas unitarias y dinámicas, la revisión del código y otras técnicas de verificación y validación.

4. Aplicación práctica sobre el caso (parte descriptiva)

Se describe cómo se usaría SonarQube en un proyecto simulado que representa el software de control de un equipo de radioterapia. Los hallazgos presentados son ilustrativos: describen el tipo de alerta que la herramienta emite ante cada patrón de defecto, y no corresponden a una ejecución real ni reproducen identificadores exactos de reglas.

4.1 Módulo simulado

El siguiente fragmento en lenguaje C condensa tres defectos inspirados en el caso: una variable compartida entre el manejador del teclado y la tarea de configuración sin ningún mecanismo de exclusión, un contador de un byte que al desbordarse omite una verificación de seguridad, y una condición que descarta silenciosamente un error.

Figura 2

Módulo simulado con defectos inspirados en el Therac-25

```
/* Modulo simulado - ilustrativo */ volatile int modo_tratamiento; /* escrito por
teclado, leído por config */ unsigned char contador_pasadas = 0; /* 1 byte: se
desborda al llegar a 256 */

void manejador_teclado(int nuevo_modo) { modo_tratamiento =
nuevo_modo; /* (A) sin sincronizacion */ }

void tarea_configuracion(void) { contador_pasadas++; /* (B) vuelve a 0
tras 255 */ if (contador_pasadas != 0) { /* (C) con 0 se omite la verificacion */
verificar_posicion_colimador(); }
configurar_haz(modo_tratamiento); /* (A) lectura de dato compartido */ }

int aplicar_dosis(void) { int estado = leer_estado_equipo(); if (estado ==
ERROR_54) { return 0; } /* (D) error ignorado sin registro */ return emitir_haz();
}
```

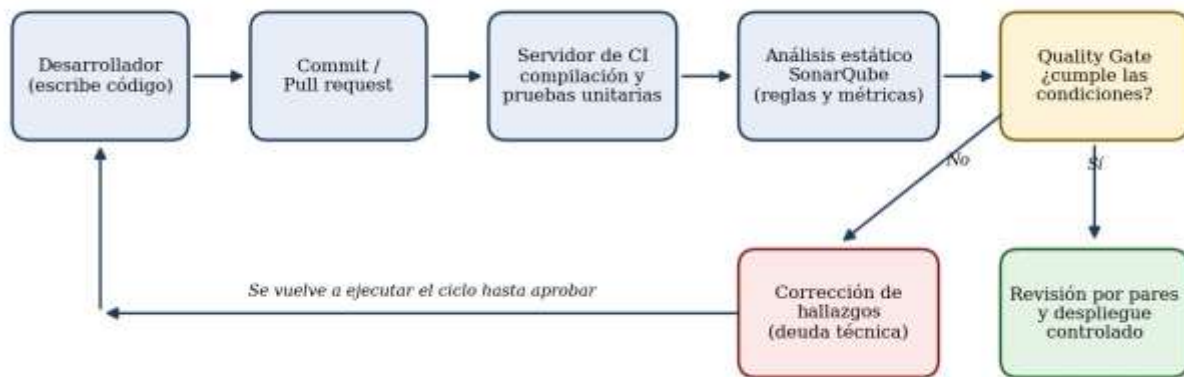
Nota. Código de elaboración propia con fines ilustrativos; no es el código original del Therac-25.

4.2 Proceso de calidad con la herramienta

El proceso se integra al flujo de desarrollo como muestra la Figura 3: cada cambio de código pasa por compilación y pruebas unitarias, luego por el análisis estático y finalmente por el quality gate. Si el gate falla, el cambio vuelve al desarrollador y no se despliega hasta que los hallazgos se corrijan. Este esquema se apoya en la integración continua, que Humble y Farley (2010) proponen como el medio para recibir retroalimentación rápida sobre cada cambio mediante procesos automatizados.

Figura 3

Flujo de SonarQube dentro del proceso de desarrollo



Nota. Elaboración propia.

4.3 Hallazgos esperados y su relación con las fallas del caso

Tabla 4

Relación entre las fallas del Therac-25, los hallazgos esperados y las medidas de calidad

Falla del caso	Hallazgo esperado en el análisis (ilustrativo)	Medida correctiva y complementaria
F1. Condición de carrera (marca A)	Alerta por acceso a una variable compartida sin sincronización; posible bug de concurrencia en la lectura y escritura.	Proteger el dato con exclusión mutua o pasar mensajes entre tareas; agregar pruebas de concurrencia y de estrés con entradas rápidas.
F2. Desbordamiento (marcas B y C)	Alerta por aritmética que puede desbordar el tipo y por una condición que depende del valor envuelto del contador.	Usar un indicador booleano explícito en lugar de un contador; pruebas de valores límite; revisión de código.

F3. Falta de interbloqueos	No es detectable por análisis estático, porque es una decisión de diseño.	Análisis de peligros y riesgos en el diseño; mantener enclavamientos por hardware independientes del software.
F4. Reutilización sin revisión	Métricas de complejidad, duplicación y deuda técnica elevadas en el código heredado.	Revisar y probar el código reutilizado en su nuevo contexto antes de integrarlo.
F5. Errores crípticos y manejo (marca D)	Alerta por manejo de excepción o estado de error ignorado sin registro.	Registrar los errores y mostrar mensajes comprensibles para el operador; pruebas de usabilidad.
F6. Pruebas insuficientes	Cobertura baja en código nuevo; el quality gate falla.	Exigir cobertura mínima, pruebas de integración y revisión independiente; gestionar casos y defectos con trazabilidad.

Nota. Los hallazgos describen el tipo de alerta esperado y no una salida real de la herramienta. Elaboración propia a partir de Leveson y Turner (1993).

5. Evaluación de impacto y medidas anticipadas

5.1 Impacto en la resolución de problemas

De las seis fallas de la Tabla 4, SonarQube aporta una señal directa y temprana en cuatro de ellas (F1, F2, F5 y F6) y una señal indirecta en F4, mediante las métricas de deuda técnica. En F3 no aporta, ya que se trata de una decisión de arquitectura. El impacto principal es de proceso: el quality gate convierte la calidad del código en una condición verificable antes de cada integración, de modo que la omisión de pruebas o la reutilización sin revisión, que en el caso pasaron inadvertidas, quedarían visibles para el equipo y para los responsables.

Esta evaluación coincide con lo que señalan Leveson y Turner (1993): ninguna herramienta aislada habría evitado los accidentes, pues la causa de fondo fue una cultura de seguridad deficiente que dependía de una sola capa de protección. Por eso el valor de la herramienta está en integrarse a un sistema de defensas en profundidad. En esa línea, Leveson (1995) plantea que la seguridad debe tratarse como una propiedad del sistema completo, de modo que el análisis de peligros precede y complementa cualquier verificación del código.

5.2 Medidas anticipadas

Con base en el análisis, se proponen las siguientes medidas para un proyecto de software crítico como el del caso:

- **Quality gate estricto.** Exigir cero bugs y cero vulnerabilidades nuevas, calificaciones máximas de fiabilidad y seguridad, cobertura de pruebas sobre el código nuevo de al menos 80 % y duplicación baja. La configuración por defecto de SonarQube (Sonar way) exige, entre otras condiciones, una cobertura mínima de 80 % en el código nuevo (SonarSource, s. f.).
- **Integración continua obligatoria.** Ningún cambio se integra sin pasar compilación, pruebas y análisis estático.
- **Pruebas dinámicas complementarias.** Pruebas unitarias con JUnit o equivalentes, pruebas de concurrencia, de valores límite y de integración del sistema completo.
- **Revisión por pares independiente.** Revisión humana del código crítico y del código reutilizado, como complemento de la herramienta. Fagan (1976) mostró que las inspecciones formales de diseño y código permiten detectar errores en etapas tempranas del desarrollo.
- **Análisis de riesgos y seguridad por diseño.** Identificación de peligros desde el diseño y mantenimiento de interbloqueos por hardware, siguiendo marcos como la norma IEC 62304, que define los procesos del ciclo de vida del software de dispositivos médicos (International Electrotechnical Commission [IEC], 2015).
- **Gestión de pruebas y defectos.** Registro con trazabilidad entre requisitos, pruebas e incidentes, con una respuesta formal a cada falla reportada por los usuarios.
- **Comunicación clara al operador.** Mensajes de error descriptivos y procedimientos ante alarmas.

5.3 Riesgos de la propuesta

Los principales riesgos son confiar en exceso en el semáforo verde del quality gate, ignorar hallazgos por considerarlos falsos positivos sin analizarlos, y configurar umbrales tan laxos que pierdan utilidad. Se mitigan con revisión periódica de las reglas, con registro de las excepciones aprobadas y con la combinación de la herramienta con pruebas dinámicas y revisión humana.

Conclusiones

Al comparar los cinco programas, es evidente que cada uno de ellos tiene una finalidad diferente y ofrece herramientas específicas y análisis distintos de los utilizados en el caso del Therac-25. SonarQube parecía ser el programa que más se adaptaba a este caso porque detecta defectos de diseño en el código, realiza un análisis estático del código y elimina las brechas de seguridad y los errores de programación mediante la implementación de compuertas de calidad. Las demás herramientas se orientan principalmente a diferentes

tipos de pruebas. Por ejemplo, el uso del analizador de código puede parecer más eficiente que las demás aplicaciones en el proceso de pruebas de funciones específicas del Therac-25. JUnit puede emplearse para evaluar si una parte específica de código concreta funciona correctamente. Al mismo tiempo, JMeter parece ser la herramienta menos pertinente para esta tarea, ya que se dirige a pruebas de carga y de rendimiento. Por último, Jenkins parece ser una herramienta de integración y automatización, capaz de ejecutar pruebas y análisis. Por lo tanto, es evidente que SonarCube es la herramienta que parece ser la más eficiente para el caso concreto del Therac-25. La razón principal por la que esta aplicación encaja mejor que las demás es que varios de los problemas y riesgos del Therac-25 están directamente relacionados con la ambigüedad del código y el proceso de desarrollo. Por lo tanto, si bien JUnit y el sistema integrado de Jenkins pueden ayudar a verificar si un código específico funciona correctamente, los análisis estáticos ofrecidos por SonarCube proporcionarían una forma más fiable de garantizar que no existan riesgos y que se haya implementado correctamente. En cuanto a Selenium y JMeter, su aplicación ofrece pocas oportunidades de mejorar el Therac-25, ya que están orientadas y están diseñadas principalmente para el análisis de aplicaciones web y el rendimiento del sistema, respectivamente. La descripción ofrecida de las peculiaridades y capacidades de cada herramienta permitió comprender plenamente sus diferencias y su funcionamiento. Por ejemplo, al utilizar SonarCube, se pueden detectar problemas relacionados con la imposición insegura de objetos compartidos, desbordamientos, excepciones y una cobertura y calidad deficientes del código. Mediante JUnit, es posible verificar que una función o un código específico funcione de la manera prevista usando casos concretos. Por último, con Jenkins, estos casos podrían integrarse en el sistema y ejecutarse automáticamente cada vez que se envíe código nuevo para su verificación. No obstante, para mejorar la codificación y garantizar que no se hayan introducido errores de diseño, un sistema automatizado no es suficiente ni totalmente preciso. Es posible que SonarCube no sea capaz de determinar si, por ejemplo, algunas partes específicas del código requerirían una verificación dinámica. Se necesitan múltiples herramientas para verificar diferentes aspectos de un sistema o aplicación.

Por lo tanto, podría decirse que, en el caso del Therac-25, no existe una única herramienta suficiente, mientras que se necesitan varias para garantizar que no haya errores en el código y que este funcione correctamente. Todas las cinco herramientas tienen una función específica que puede ser beneficiosa para el sistema analizado. Por ejemplo, para verificar y garantizar la seguridad del Therac-25 y eliminar la posibilidad de errores de codificación y de diseño, aplicarías SonarCube, JUnit, la automatización de pruebas y verificación de Jenkins, y Selenium, si el sistema incluyera una interfaz web. Por último, al integrar JMeter en el proceso de pruebas y verificación, es posible asegurarse de que no habrá problemas de rendimiento. Los resultados y las conclusiones descritos anteriormente en la explicación del caso analizado por Leveson y Turner en 1993 son relevantes y adecuados para el Therac-25 porque un solo sistema y una sola herramienta de verificación no son ni adecuados ni suficientes.

Referencias

- Aizprua, S., Ortega, A., & Von Chong, L. (2019). Calidad del software, una perspectiva continua. *Revista Científica Universitaria*, 8(2), 1–18.

- Campbell, G. A., & Papapetrou, P. P. (2013). *SonarQube in action*. Manning Publications.
- Cunningham, W. (1992). The WyCash portfolio management system. En *OOPSLA '92 addendum to the proceedings on object-oriented programming systems, languages, and applications* (pp. 29–30). ACM. <https://doi.org/10.1145/157709.157715>
- Fagan, M. E. (1976). Design and code inspections to reduce errors in program development.
 - *IBM Systems Journal*, 15(3), 182–211. <https://doi.org/10.1147/sj.153.0182>
- Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.
- International Electrotechnical Commission. (2015). *Medical device software — Software life cycle processes* (IEC 62304:2006+AMD1:2015).
- International Organization for Standardization & International Electrotechnical Commission.
 - (2023). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC 25010:2023).
- JUnit. (s. f.). *JUnit 5 user guide*. <https://junit.org/junit5/docs/current/user-guide/>
- Leveson, N. G. (1995). *Safeware: System safety and computers*. Addison-Wesley.
- Leveson, N. G., & Turner, C. S. (1993). An investigation of the Therac-25 accidents. *Computer*, 26(7), 18–41. <https://doi.org/10.1109/MC.1993.274940>
- McCall, J. A., Richards, P. K., & Walters, G. F. (1977). *Factors in software quality* (RADC-TR-77-369). Rome Air Development Center.
- Myers, G. J., Sandler, C., & Badgett, T. (2011). *The art of software testing* (3.^a ed.). John Wiley & Sons.
- Piattini-Velthuis, M., & García-Rubio, F. (2015). *Calidad de sistemas de información*.
 - RA-MA Editorial.
- Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería de software*. McGraw-Hill Interamericana.
- Selenium. (s. f.). *Selenium documentation*. <https://www.selenium.dev/documentation/>
- SmartBear Software. (2024). *SoapUI* [Sitio web]. <https://www.soapui.org>
- SonarSource. (s. f.). *SonarQube documentation*. <https://docs.sonarsource.com/sonarqube/>